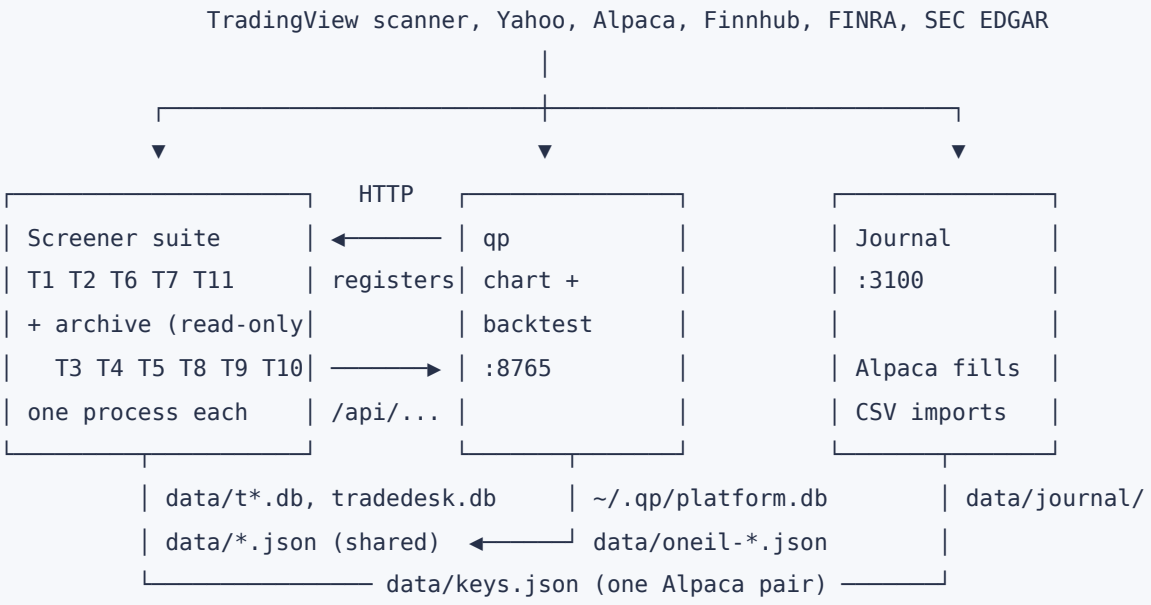
Architecture: the desk as it runs today

How the three apps work and how they depend on each other. Read this before changing anything: most past damage came from a change that was right for one file and wrong for the system around it.

The design for building it again from the beginning is REBUILD-ARCHITECTURE.md.



Every app reads `tools.config.json` (the registry) and serves the same top bar from `public/desk.js` + `public/desk.css`.

One login guards every app — `src/auth/index.js` (Node: tools, archive, journal) and `quant-platform/chart/auth.py` (qp, pure ASGI so its WebSocket is covered) check the same `data/auth.json` and the same `td_auth` cookie, which a browser shares across ports. Calls from the box itself (127.0.0.1, no proxy headers) pass, so the apps' calls to each other, the backup and the deploy's health checks need no cookie. Pages reading another port send the cookie (`desk.js` wraps `fetch`); the landing probe paths answer with credentialed CORS for the same host only.

Every night the databases and register CSVs go to Cloudflare R2 (the backup is documented with the system-wide pages).

Screener suite (`src/`)

One process per tool. `src/index.js` with `TOOL_ID` and its own `DB_PATH` (`data/t2.db`, …; T1 keeps `data/tradedesk.db`). Each tool has its own screeners (TradingView filter sets stored in its

DB), its own scan window and its own capture times (`captureAt` in the registry). Tools added from the screen live in `data/tools.local.json`, merged by `src/toolsRegistry.js`.

The scan (`src/pipeline.js`, `runFullScan`), in order:

Side	Folder	Fatal?	What
A	`sideA/`	yes	TradingView scanners → merged rows; pre-R0 enrichment
B	`sideB/`	yes	Derived fields (gap %, ADR, range position, …)
—	`sideC/shortInterest`, `sideC/borrow`, `sideA/canslimRow`, `sideA/industryMap`	no	Soft enrichment on this scan's rows, before the upsert
D	`sideD/`	no	Market context: regime, sector bias, hot sectors, O'Neil market read. Needs SPY or QQQ with its moving averages, or the stage fails rather than guessing
E	`sideE/`	no	Python scorer — off on every tool today (`scorer: false`), so `_score` is null
G	`sideG/staleFetch`	no	Refresh quotes for names not in this scan
C	`sideC/`	no	News and catalysts
F	`sideF/`	no	Shortlist flags restored from the DB

Rows live in **r0** (`src/r0/registry.js`): an in-memory map checkpointed to SQLite, flushed at midnight.

The card is the biggest part of the screener tools. TradingView supplies the candidates and their 32 columns; everything else on a card comes from nine more sources and the desk's own computations, each with its own cache, freshness and fallback:

On the card	Sources
quote, levels, ranges, volume, float, cap	TradingView scanner (re-quoted every 5 min)
derived numbers, relations, technical catalysts	computed
short interest, days to cover	FINRA → Yahoo → Nasdaq
borrow	Alpaca (shortable), Nasdaq + NYSE (Reg SHO lists)
news, catalyst, auto bias	TradingView news, Alpaca (Benzinga), Yahoo, SEC EDGAR, Finnhub; Google News as fallback
market context, sector heat	the market snapshot (TradingView: indexes and 15 sector ETFs), computed by each tool
industry and group	SIC (EDGAR) first, TradingView label otherwise
the CANSLIM reading (C, A, I, S, N, L, M, base, ratings)	O'Neil values computed by qp (EDGAR facts, 13F, Polygon, Yahoo bars)
score, shortlist marks	the tool's scorer; this tool's and every tool's stars

Stage by stage, field by field: DATA-PIPELINE.md, CARD.md, CARD-FIELDS.md.

The Market tab has two halves: the desk's live read (indexes, regime, sector heatmap, hot sectors), computed by each tool on every scan from TradingView's index and sector-ETF quotes; and O'Neil's method (market model M, industry groups L), computed by qp after each close. The live read is also stamped on every card. See MARKET-TAB.md.

Schedule (`src/scheduler.js`, every job in America/New_York, weekdays): discovery scans every 30 min 04:00–09:00, every 5 min 09:00–10:00, every 15 min 10:00–16:00, once at 16:00; a live-card refresh every 5 min 04:00–16:00; the shortlist auto-rule, R1 capture and R2 snapshots at each tool's capture times; R3 end-of-day capture 16:05; scorer training 16:20; r0 flush at midnight. The R2 backup (`scripts/backup-r2.js`) runs under pm2 at 23:20 UTC.

Registers (`src/warehouse/`): R1 (the frozen card set at the tool's R1 time), R2 (market snapshots), R3A/R3B (entry A/B outcomes from Alpaca bars), R4A/R4B (R1 fields joined with outcomes — the research and training set), Shortlist. qp reads these over HTTP for charts and backtests.

Archive (`src/archive/server.js`): one read-only process serving the stored registers of the stopped tools on their own ports, so qp can still chart and backtest everything they collected.

Pages (`public/`): `home.html` (landing), `index.html` (the tool app: screener, market, shortlist, warehouse, analysis, screeners, settings, monitor), `screeners.html` (suite, `/screeners`), `manage.html` (Manage tools, `/screeners/manage` — its controls are T1's `/api/tools/manage`), `compare.html` (`/screeners/compare`). PAGES.md walks every screener page.

The scanner's data — sources, processing, card and registers — is in DATA-PIPELINE.md, REGISTERS.md, CARD.md, CARD-FIELDS.md and MARKET-TAB.md. The intent for the score, the screener builder and the scorecard, ahead of their rebuild, is in IDEAS.md.

Running a tool — settings (SETTINGS.md), the scheduled jobs (MONITOR.md), the data-source check (DATACHECK.md), adding and stopping tools (TOOL-LIFECYCLE.md) and every route (API.md).

qp (`quant-platform/`)

- `qp/primitives/` — the only place market math exists; every indicator is a registered, TradingView-verified primitive (`approvals/approvals.json`).
- `chart/server.py` — FastAPI app: chart data and the live WebSocket, strategies, backtests (run, report, CSV), screener register browsing, O'Neil/CANSLIM reads, data checks, a signal watcher (`chart/alerts.py`).
- `chart/strategy.py` — evaluates a strategy document over bars; `chart/backtest.py` runs it day by day over a register universe; `chart/order_plan.py` sizes every order (one function: share count, caps, minimum shares, whole-share legs, cents); `chart/exit_protocol.py` turns a strategy's exits into legs.
- `tools/data/` — bar loaders (yahoo, alpaca, polygon, hybrids) with a parquet cache in `~/.qp-cache/`.
- Strategies and backtests live in `~/.qp/platform.db` (`QP_DB` overrides); `chart/seeds/*.json` are the git copies, restored when missing.
- **qp hosts the O'Neil computations, but their results belong to the screener tools**: the card's CANSLIM fold, the `cs*` columns frozen into R1, and the Market tab's M and L. A nightly job (systemd `deploy/qp-daily.timer`, 22:30 UTC) rebuilds RS, the market model, groups, SIC, 13F and EDGAR into `data/oneil-*.json` and `data/industry-map.json`; each scan also asks qp over HTTP for the per-card values, so a card has no CANSLIM reading while qp is down (CANSLIM.md).

Journal (`journal/`)

Express app, `journal/server.js`, data in `data/journal/` (`JOURNAL_DATA_DIR`). Imports broker/CSV files; the accounts add-on (`journal/src/addons/`) stores named Alpaca accounts (mode 0600) and fetches each one's fills on its own schedule, pairing them into round-trip trades under a stable id (a closed trade is never rewritten). Grading and card logic are in `journal/src/journal/` and `journal/src/trading/`.

Shared state — the coupling to respect

What	Written by	Read by
`tools.config.json` (repo)	people	every app, the deploy
`data/tools.local.json`	screener UI (Manage tools)	every app
`data/keys.json`	by hand / `scripts/set-alpaca-keys.js` (the Settings tab has no key inputs)	screeners, journal (fallback), `scripts/sync-qp-env.js` → `quant-platform/.env`
`data/oneil-*.json`, `data/industry-map.json`	qp nightly jobs. The industry map is SIC (market-wide); screeners add TradingView labels only for symbols SIC has not classified, never over a SIC entry	screeners, qp's group ranks
`data/shortlist-all.json`	screeners	screeners
`data/auth.json` (the login, secret)	`scripts/set-password.sh`	every app (`src/auth`, `chart/auth.py`)
screener registers	screeners (SQLite)	qp over HTTP (`chart/screener.py`, ports from the registry)

Operations

pm2 supervises every long-running process; `./deploy-tools.sh` pulls, stops, starts and health-checks everything (the deploy is documented with the system-wide pages). Nothing may be supervised by systemd as well.

Where things live, and what moves them

Everything a deployment elsewhere would need to point somewhere else already has an environment variable; the defaults are the box's layout.

Thing	Default	Override
A tool’s database	`data/tradedesk.db` (T1), `data/t<N>.db`	`DB_PATH`
Shared screener files (`keys.json`, `oneil-*.json`, …)	`<repo>/data/`	per file: `ONEIL_GROUPS_FILE`, `SHARED_KEYS_FILE`, …
qp strategies + backtests	`~/.qp/platform.db`	`QP_DB`
qp bar and data caches	`~/.qp-cache/`	`QP_CACHE_DIR` (each cache also has its own, e.g. `QP_CANSLIM_CACHE`)
qp operations log	beside the code	`QP_DATA_DIR`
Journal data	`<repo>/data/journal/`	`JOURNAL_DATA_DIR`, `JOURNAL_DB`
qp → screeners	ports from `tools.config.json` on `http://localhost`	`TOOLS_HOST`, `SCREENER_SOURCES`, `SCREENER_URL`
screeners → qp	`http://127.0.0.1:8765`	`QP_URL`
screeners → scorer	`http://127.0.0.1:3001`	`SCORER_URL`
HTTP port	from the registry	`PORT`

Box-only by design: the systemd units in `quant-platform/deploy/` (user `ec2-user`, `/home/ec2-user/Tade-desk-server`) and the deploy script’s one-time copy from `~/journal-app`.